ОТЧЁТ

по лабораторной работе №21

«Итоговый проект: система управления задачами»

Вариант 20. Система управления задачами

1. Цель работы

Собрать законченное приложение, в котором есть модель предметной области, хранение, HTTP-интерфейс, проверка тестами и описание запуска в контейнере.

2. Постановка индивидуального задания

Вариант 20. Реализовать User, Project, Task и Comment. У задачи должны быть приоритет, статус, исполнитель и дедлайн.

3. Краткие теоретические сведения

Итоговая система соединяет приёмы курса: сущности и связи SQLAlchemy, проверки входных данных FastAPI, страницу со списком и тесты. Комментарий относится к задаче, задача — к проекту и исполнителю. Это композиция объектов, а не один общий словарь.

Dockerfile фиксирует способ запуска через uvicorn. Тесты создают отдельную базу во временном каталоге, чтобы не портить учебные данные сервера.

4. Описание программы

Пользователь имеет имя, проект — название, задача — название, приоритет, статус, дедлайн, проект и исполнителя, комментарий — текст и автора. Стартовые данные: Иванов и Петрова, проекты «Сайт провайдера» и «Биллинг», две задачи с дедлайнами.

POST /tasks создаёт задачу только при существующем проекте и исполнителе и при приоритете из набора «низкий», «средний», «высокий». Отдельные адреса меняют статус и добавляют комментарий. Страница / показывает таблицу. Тесты создают задачу с дедлайном, добавляют комментарий и проверяют, что название видно на странице.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Листинг — app.py

```
"""
Лабораторная работа №21. Вариант 20.
Система задач: пользователь, проект, задача, комментарий.
"""

import os

from fastapi import FastAPI, HTTPException
from fastapi.responses import HTMLResponse
from pydantic import BaseModel
from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session

from models import Base, Comment, Project, Task, User

BASE = os.path.dirname(os.path.abspath(__file__))
ENGINE = create_engine(f"sqlite:///{os.path.join(BASE, 'data', 'tasks.db')}", echo=False)



class TaskIn(BaseModel):
    title: str
    project_id: int
    assignee_id: int
    priority: str = "средний"
    deadline: str = ""



class StatusIn(BaseModel):
    status: str



class CommentIn(BaseModel):
    text: str
    author: str



def seed(session: Session):
    if session.scalar(select(User).limit(1)):
        return
    ivan = User(name="Иванов")
    anna = User(name="Петрова")
    session.add_all([ivan, anna])
    site = Project(title="Сайт провайдера")
    billing = Project(title="Биллинг")
    session.add_all([site, billing])
    session.flush()
    session.add(Task(
        title="Макет главной", priority="высокий", status="новая",
        deadline="2026-10-01", project=site, assignee=anna,
    ))
    session.add(Task(
        title="Списание абонентской платы", priority="средний", status="в работе",
        deadline="2026-09-30", project=billing, assignee=ivan,
    ))
    session.commit()



def task_dict(task: Task):
    return {
        "id": task.id,
        "title": task.title,
        "priority": task.priority,
        "status": task.status,
        "deadline": task.deadline,
```

```
        "project": task.project.title,
        "assignee": task.assignee.name if task.assignee else "",
        "comments": [{"author": item.author, "text": item.text} for item in task.comments],
    }



def create_app(engine=None):
    db = engine or ENGINE
    os.makedirs(os.path.join(BASE, "data"), exist_ok=True)
    Base.metadata.create_all(db)
    with Session(db) as session:
        seed(session)
    app = FastAPI(title="Система задач")

    @app.get("/", response_class=HTMLResponse)
    def page():
        with Session(db) as session:
            tasks = session.scalars(select(Task)).all()
            rows = "".join(
                f"<tr><td>{task.id}</td><td>{task.title}</td><td>{task.project.title}</td>"
                f"<td>{task.assignee.name if task.assignee else ''}</td>"
                f"<td>{task.priority}</td><td>{task.status}</td><td>{task.deadline}</td></tr>"
                for task in tasks
            )
        return f"""<!DOCTYPE html><html lang="ru"><head><meta charset="utf-8"><title>Задачи</title>
        <style>body{{font-family:Segoe UI,sans-serif;margin:32px}}table{{border-
collapse:collapse;width:100%}}
        td,th{{border-bottom:1px solid #ddd;padding:8px;text-
align:left}}th{{background:#16324f;color:white}}</style>
        </head><body><h1>Система управления задачами</h1>
        <p>Пользователь, проект, задача и комментарий. У задачи есть приоритет, статус, исполнитель и
дедлайн.</p>

<table><tr><th>ID</th><th>Задача</th><th>Проект</th><th>Исполнитель</th><th>Приоритет</th><th>Статус</th><
th>Дедлайн</th></tr>
        {rows}</table></body></html>"""

    @app.get("/tasks")
    def list_tasks():
        with Session(db) as session:
            tasks = session.scalars(select(Task)).all()
            return [task_dict(task) for task in tasks]

    @app.post("/tasks", status_code=201)
    def create_task(payload: TaskIn):
        with Session(db) as session:
            project = session.get(Project, payload.project_id)
            assignee = session.get(User, payload.assignee_id)
            if project is None or assignee is None:
                raise HTTPException(400, "проект или исполнитель не найден")
            if payload.priority not in {"низкий", "средний", "высокий"}:
                raise HTTPException(400, "приоритет: низкий, средний или высокий")
            task = Task(
                title=payload.title, priority=payload.priority, deadline=payload.deadline,
                project=project, assignee=assignee,
            )
            session.add(task)
            session.commit()
            session.refresh(task)
            return task_dict(task)

    @app.post("/tasks/{task_id}/status")
    def set_status(task_id: int, payload: StatusIn):
        with Session(db) as session:
            task = session.get(Task, task_id)
            if task is None:
                raise HTTPException(404, "задача не найдена")
            task.status = payload.status
            session.commit()
            return task_dict(task)

    @app.post("/tasks/{task_id}/comments", status_code=201)
    def add_comment(task_id: int, payload: CommentIn):
        with Session(db) as session:
            task = session.get(Task, task_id)
```

```
            if task is None:
                raise HTTPException(404, "задача не найдена")
            session.add(Comment(text=payload.text, author=payload.author, task=task))
            session.commit()
            session.refresh(task)
            return task_dict(task)

    return app


app = create_app()
```

Листинг — models.py

```
"""Итоговая модель: пользователь, проект, задача, комментарий."""

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    tasks: Mapped[list["Task"]] = relationship(back_populates="assignee")


class Project(Base):
    __tablename__ = "projects"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    tasks: Mapped[list["Task"]] = relationship(back_populates="project")


class Task(Base):
    __tablename__ = "tasks"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    priority: Mapped[str] = mapped_column(String(20), default="средний")
    status: Mapped[str] = mapped_column(String(30), default="новая")
    deadline: Mapped[str] = mapped_column(String(20), default="")
    project_id: Mapped[int] = mapped_column(ForeignKey("projects.id"))
    assignee_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"), nullable=True)
    project: Mapped[Project] = relationship(back_populates="tasks")
    assignee: Mapped[User | None] = relationship(back_populates="tasks")
    comments: Mapped[list["Comment"]] = relationship(back_populates="task")


class Comment(Base):
    __tablename__ = "comments"
    id: Mapped[int] = mapped_column(primary_key=True)
    text: Mapped[str] = mapped_column(String(300))
    author: Mapped[str] = mapped_column(String(100))
    task_id: Mapped[int] = mapped_column(ForeignKey("tasks.id"))
    task: Mapped[Task] = relationship(back_populates="comments")
```

Листинг — requirements.txt

```
fastapi==0.141.1
uvicorn==0.30.6
sqlalchemy==2.1.1
httpx==0.28.1
pytest==9.1.1
```

Листинг — tests\test_tasks.py

```
import os
import sys
```

```
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

from app import create_app
from models import Base, Project, User


def client(tmp_path):
    engine = create_engine(f"sqlite:///{tmp_path / 't.db'}")
    Base.metadata.create_all(engine)
    with Session(engine) as session:
        session.add(User(name="Иванов"))
        session.add(Project(title="Сайт"))
        session.commit()
    return TestClient(create_app(engine))


def test_create_with_priority_and_deadline(tmp_path):
    api = client(tmp_path)
    response = api.post("/tasks", json={
        "title": "Вёрстка", "project_id": 1, "assignee_id": 1,
        "priority": "высокий", "deadline": "2026-10-05",
    })
    assert response.status_code == 201
    body = response.json()
    assert body["assignee"] == "Иванов"
    assert body["deadline"] == "2026-10-05"
    assert body["priority"] == "высокий"


def test_comment_and_status(tmp_path):
    api = client(tmp_path)
    created = api.post("/tasks", json={
        "title": "API", "project_id": 1, "assignee_id": 1, "priority": "средний",
    }).json()
    status = api.post(f"/tasks/{created['id']}/status", json={"status": "в работе"})
    comment = api.post(f"/tasks/{created['id']}/comments", json={"text": "взял в работу", "author":
"Иванов"})
    assert status.json()["status"] == "в работе"
    assert comment.json()["comments"][0]["text"] == "взял в работу"


def test_page_lists_task(tmp_path):
    api = client(tmp_path)
    api.post("/tasks", json={"title": "Отчёт", "project_id": 1, "assignee_id": 1})
    page = api.get("/")
    assert page.status_code == 200
    assert "Отчёт" in page.text
```

6. Примеры работы программы

Снимки показывают страницу запущенной системы и результат тестов.

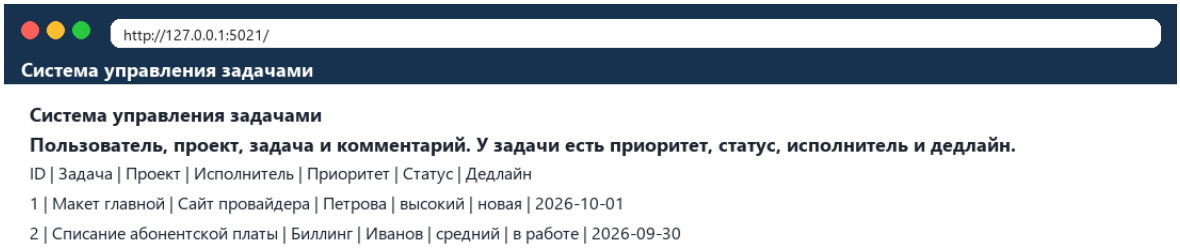


Рисунок 1 – Страница задач: исполнитель, приоритет, статус и дедлайн

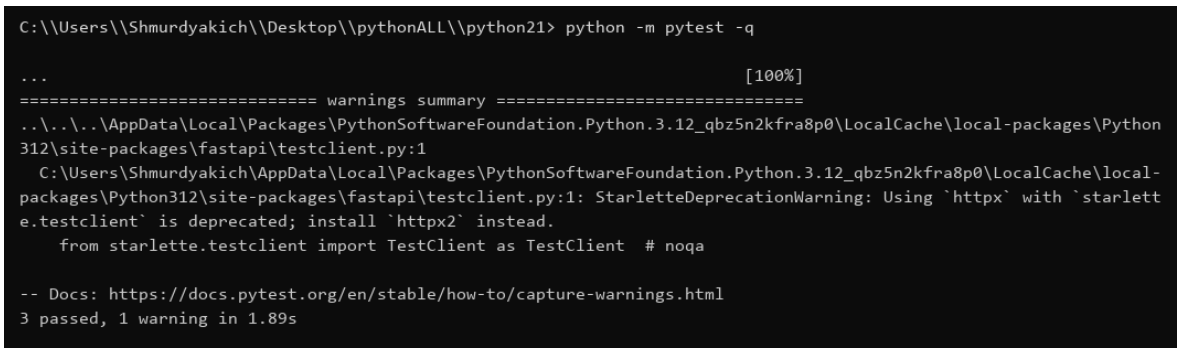


Рисунок 2 – Тесты создания задачи, комментария и страницы

7. Ответы на контрольные вопросы

1. Какие сущности входят в вариант?

Пользователь, проект, задача и комментарий. У задачи есть приоритет, статус, исполнитель и дедлайн.

2. Как задача связана с остальными?

Она принадлежит проекту и назначается пользователю. Комментарии ссылаются на задачу.

3. Где проверяется приоритет?

В обработчике создания. Значение вне трёх допустимых отклоняется с кодом 400.

4. Зачем отдельная база в тесте?

Чтобы проверка создания задачи не зависела от данных, которые уже открыл сервер.

5. Что даёт контейнер итоговому проекту?

Одинаковый запуск: образ ставит зависимости и стартует uvicorn на порту 8000.

8. Вывод

В итоговой работе варианта 20 собрана система задач: пользователь, проект, задача и комментарий хранятся в SQLite, отдаются через FastAPI и видны на странице. У задачи есть приоритет, статус, исполнитель и дедлайн. Тесты и Dockerfile фиксируют проверку и способ запуска.